Backpressure

let a slow consumer tell fast producers to slow down

Watch the 4-min video

In one line: backpressure is a slow consumer signalling producers to slow down: bound every queue and buffer, and when full block (in-process), reject (429/503 with Retry-After across HTTP) or drop (telemetry); use protocol flow control (TCP windows, HTTP/2 and gRPC, reactive streams, Kafka pull), propagate the signal to the source, and never hide overload in an unbounded queue.

1 No way to say "slow down"

Producers send jobs faster than your workers can handle them. The queue grows to millions of items. Memory fills up, latency is now hours, and finally everything crashes. The system had no way to say: *slow down*. That signal is **backpressure**.

2 What backpressure is

A slow consumer signals fast producers to slow down, so the system stays within its capacity. Without it, an **unbounded queue just hides the overload** until it runs out of memory. With it, overload is visible early and handled on purpose: a hidden failure becomes an explicit, controlled response.

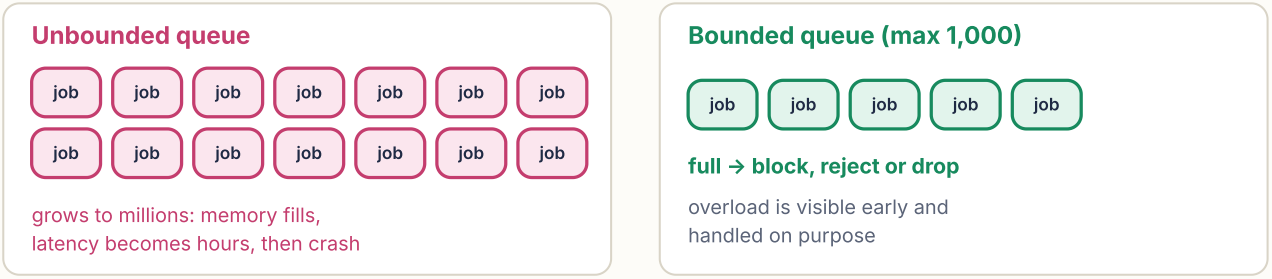


Figure 1. An unbounded queue hides overload until it fails; a bounded one forces a decision while the system is still healthy.

3 Bounded queues and the three choices

Give **every** queue and buffer a maximum size. When it's full, choose:

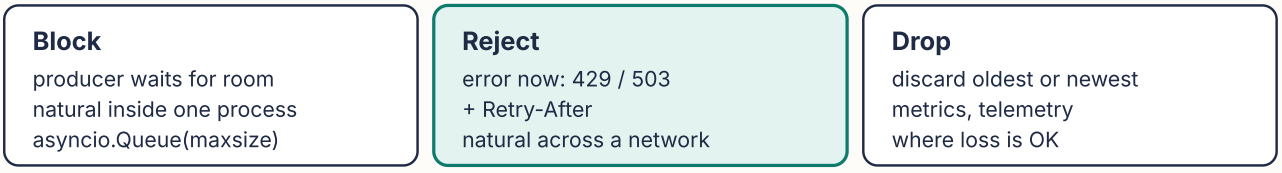


Figure 2. When a bounded queue is full: block the producer, reject the item, or drop data.

Blocking is natural inside one process (an `asyncio.Queue(maxsize=1000)` makes `await q.put()` wait). Rejecting is natural across a network. Dropping fits metrics and telemetry, where the newest data matters most.

4 Over HTTP: 429 and 503

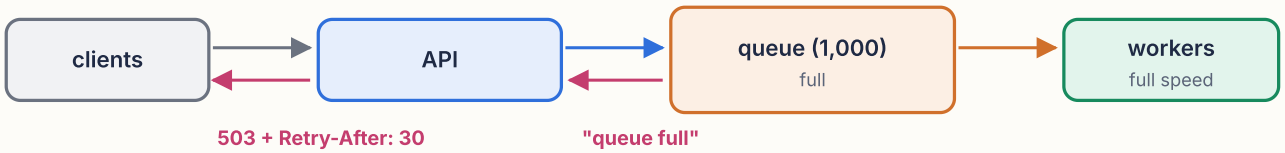
Return **429 Too Many Requests** when a client exceeds its limit, or **503 Service Unavailable** when the server is overloaded. Add a **Retry-After** header so well-behaved clients wait, ideally with exponential backoff and jitter (lesson 10 and Phase 2's API lessons on retries).

5 Built into many protocols

- **TCP flow control**: the receiver advertises a smaller window when it's slow (Phase 2, lesson 10).
- **HTTP/2 and gRPC**: per-stream flow-control windows.
- **Reactive streams**: the consumer requests exactly N items at a time.
- **Kafka**: consumers pull at their own pace; lag grows instead of memory.
- **Python**: `asyncio.Queue(maxsize)`, bounded thread-pool queues, semaphores for concurrency limits.

WORKED EXAMPLE: AN IMAGE JOB POOL

A worker pool processes image jobs from a queue limited to **1,000** items. When the queue is full, the API stops accepting new jobs and returns **503 with Retry-After: 30**. Clients back off and try again later. Workers keep running at full speed, memory stays flat, and accepted jobs still finish in minutes rather than hours.



the signal travels back to the source instead of a middle layer buffering forever

Figure 3. Backpressure end to end: the full queue makes the API reject, and clients slow down.

6 Design it end to end

- **Every hop needs a limit**: request concurrency, connection pools, queues, thread pools.
- **Propagate** the signal back toward the source, rather than letting a middle layer buffer forever.
- **Monitor** queue depth and rejection rate.
- **Combine with autoscaling** (lesson 14) and load shedding (lesson 10), so capacity grows while excess load is shed.

A system that says no early stays healthy under any load.

COMMON MISTAKES

- Unbounded in-memory queues "just in case".
- A middle service that retries and buffers everything, hiding overload from clients.
- Rejecting without Retry-After, so clients retry instantly and make it worse.

7 Key terms

Backpressure: consumer signals producers to slow down

Bounded queue: a queue with a maximum size

429 / 503: client over limit / server overloaded

Retry-After: when the client may try again

Flow control: protocol-level backpressure (TCP, HTTP/2)

Consumer lag: how far a Kafka consumer is behind

8 Interview questions

Q1. What is backpressure and how do you implement it? BASIC
It's a consumer signalling producers to slow down, implemented with bounded queues that block or reject when full, 429 or 503 with Retry-After over HTTP, and protocol flow control.

Q2. Why not just use an unbounded queue? BASIC
It hides overload until memory runs out and latency grows to hours.

Q3. A producer service floods a slow downstream API. What do you add? SCENARIO
A bounded queue or concurrency limit (semaphore) in front of the downstream calls, rejection or blocking when full, retries with backoff and jitter, and alerts on queue depth.

Q4. Block, reject or drop for a metrics pipeline? SCENARIO
Drop (usually the oldest samples or by sampling), because fresh metrics matter more than complete ones and blocking would stall the application.

Q5. How does Kafka provide backpressure? TRICKY
Consumers pull at their own pace, so a slow consumer simply falls behind (lag grows on durable disk) instead of being overwhelmed; producers are throttled by broker quotas or buffer limits.

Q6. Why should the signal reach the original source? TRICKY
If a middle layer absorbs the overload in its own buffer, it just moves the unbounded queue there; only the source can actually reduce the rate.

Next: Phase 4 · SD-H-001 · Distributed System Fundamentals backendroadmap.com